Extraction From MySQL's Binary Log

Explore techniques to extract data from MySQL's binary log using Python's BinLogStreamReader. Learn to filter insert, update, and delete events from specific tables and export them as CSV files for ETL processes. Understand how transaction logs help database recovery and synchronization.

Most popular relational database management system (RDBMS) solutions keep a log file of all changes to the database, including changes due to data manipulation language (DML) statements such as `INSERT`, `UPDATE`, and `DELETE`, as well as data definition language (DDL) statements such as `CREATE` and `DROP`.

Each database calls this log file by a different name, but all serve the same purpose.

- In Microsoft SQL Server, the log file is called the **transaction log**.
- In Oracle, the log file is called the **redolog.**
- In PostgreSQL, the log file is called the **write-ahead log (WAL)**.
- In MySQL, the log file is called the **binary log** or **binlog**. We'll explore this log file in detail in this lesson.

Why do we need a log file?

These log files are useful in case of database crashes and help keep the database up to date. This allows in-memory changes that have not yet materialized on disk to be reconstructed from the log file in case of a crash.

Ask

The log file is also used for syncing clusters of distributed databases. Without going into too much depth, a common architecture choice for distributed databases is leader-follower architecture, where one database node is the leader and receives all the write requests from users. The rest of the nodes are read-only databases.

When the leader receives a write request and changes a record in its version of the database, it sends its log file to the other nodes so that they can perform the same actions as the leader. This way, the whole distributed database cluster is synced again.

Another usage of this log file is that it can be a great data source in ETL pipelines for extracting log activity data about the company.

Example: Extracting data from MySQL's binary log

One way to extract log activity data from MySQL's binlog is using Python's `BinLogStreamReader`. For example, we'll connect to MySQL's binlog and extract the log activity data about the table `customers`.

The table had several insert and update queries applied to it as shown below.

main.sql MySQL

```sql
-- Create table customers
CREATE TABLE customers (
  id int not null primary key,
  first_name varchar(30),
  last_name varchar(30),
  active int,
  LastUpdated timestamp
);

-- Populate table
INSERT INTO customers
  VALUES(1, 'Bruce', 'Wayne', 1, '2022-06-01 12:00:00');
```

Ask

```
INSERT INTO customers
```

Run

Simulating activity within the customer's table - several insert and update queries

We need to connect to MySQL and specify which types of log events we would like to extract(i.e., delete event, update event, etc.), then process the raw data and export it as CSV.

Connecting to MySQL using Python's `BinLogStreamReader`

First, run the following code snippet to connect to MySQL's `Binlog` by specifying the connection details, such as `host`, `port`, `user`, and `password`.

Notice that we use Python's `BinLogStreamReader` to connect and specify the types of events we would like to extract, which are inserts (`WriteRowsEvent`), updates (`UpdateRowsEvent`), and deletes (`DeleteRowsEvent`).

main.py Python 3.10.4

```
from pymysqlreplication import BinLogStreamReader
from pymysqlreplication import row_event
import pymysql
import csv
import pandas as pd
import datetime

# Grabbing today's date
today = datetime.datetime.today()
start_date = today - datetime.timedelta(days = 1 )
timestamp = start_date.timestamp()

'''
```

Ask

Connecting to MySQL's binary log

- **Lines 25–30:** Connect to MySQL binary log. Include only events like inserts, updates, and deletes. Skip to events that occurred in the past day.

Processing log events

Next, we can iterate over the events in a for loop and process them however we need. In this code snippet, we specify that the table we are interested in is the `customers` table. And, if any event is an instance of either `WriteRowsEvent`, `UpdateRowsEvent`, or `DeleteRowsEvent`, extract it into a dictionary, and create a Pandas DataFrame of all the events in the dictionary.

main.py Python 3.10.4

```
'''
Process Log Events - Create Dictionaries with the Newly Updated Data
'''
events = []

# Itrate over the events in the binlog
for event in b_stream:
    for row in event.rows:

        # Collect delete/update/insert events from the customers table
        if event.table =='customers':
            event_dict = {}
```

Run

Processing events

- **Line 7:** We iterate over the log events.
- **Line 11:** We extract only the events related to the table `customers`.
- **Lines 15–23:** We append each event to a new dictionary and classify each event as either `delete`, `update` or, `write`.
- **Line 29:** We load the event dictionary to a pandas DataFrame.

Ask

Exporting the data to a CSV file

Finally, we export the DataFrame to a CSV file to be used later.

main.py Python 3.10.4

```
'''
Export Data to CSV
'''
if len(events) > 0:

    try:
        # Export events to CSV
        keys = events[0].keys()
        local_filename = 'customers_events.csv'
        with open(local_filename, 'w', newline='') as output_file:
            dict_writer = csv.DictWriter(output_file, keys, delimiter='|'
            dict_writer.writerows(events)
```

Run

Exporting data to a CSV file

- **Lines 10–12:** We write the events dictionary to a CSV file named `customers_events.csv`.

In this lesson, we dealt with the magnitude of transaction logs in RDBMS. These logs are like video recorders for all database changes, and this also allows crashes to be recovered easily. This regulates the synchronization among the databases.

The major point was MySQL's binary log (binlog), where we showed how to get log activity data using the `pymysqlreplication` Python library. By connecting to the MySQL binary log, processing events from the log related to a particular table, and exporting the data as CSV, the concept of applying database logs to th (extract, transform, load) pipeline has been illustrated.

Ask

Understanding and manipulating database logs through programming interfaces such as Python offers a wide array of value-added information for extracting data, monitoring, and analyzing real-world scenarios in database management.

Ask